

SOA-to-Agentic AI Terminology Mapping

28 Service-Oriented Architecture concepts and their agentic equivalents. Version 2.0.0. Related concepts are grouped together, so you can read down either column. CC-BY 4.0 · doi.org/10.5281/zenodo.21823088

SOA ERA

Service / Microservice

A discrete unit of functionality that can be accessed remotely and acted upon independently, deployed and versioned on its own so that changing it does not require changing its consumers.

Microservices

Architectural approach using small, independently deployable services, each owning a narrow slice of business capability so teams can build, release, and scale them separately.

Common Services

Reusable services that provide shared functionality across multiple applications — authentication, notification, document generation — maintained once and consumed everywhere.

Service Contract

A formal specification defining how to interact with a service, including inputs, outputs, and constraints, so consumers integrate against the contract rather than the implementation.

Service Registry

A directory where services are published and can be discovered by consumers, holding the endpoint, version, and contract for each registered service.

Service Discovery

A dynamic mechanism for locating available services at runtime, resolving which instance to call as deployments scale up and down.

AGENTIC ERA

Agent

An autonomous AI entity that can perform specific tasks, make decisions, and interact with other agents, choosing how to accomplish an objective rather than following a fixed sequence of steps.

Agent Tools

Specialized, focused AI agents designed for specific tasks within larger systems, narrow enough to be reasoned about, tested, and replaced individually rather than as part of one monolithic agent.

Agentic Tools

Reusable AI capabilities and functions that agents can utilize to perform common tasks, exposed as a shared library so every agent does not reimplement the same retrieval, extraction, or calculation.

Agent Contract

A specification defining an agent's capabilities, interaction protocols, and behavioral constraints — what it may do on its own, what it must escalate, and what callers can rely on.

Agent Catalog

A repository where agents are registered and can be discovered by other agents or orchestrators, recording each agent's capabilities, ownership, and permitted scope.

Agent Discovery

A dynamic mechanism for locating available agents and their current capabilities at runtime, so work is routed to whichever agent is actually able and permitted to handle it.

Service Bus (ESB)

Enterprise service bus providing message routing, transformation, and integration capabilities, so systems that do not share a data format can still exchange work.

API Gateway

A single entry point managing routing, security, authentication, and rate limiting for service consumers, so those concerns are handled once at the edge rather than in every service.

Orchestration (BPEL)

Coordinating multiple services to execute complex business processes, with the sequence, branching, and error handling defined centrally.

Service Choreography

A peer-to-peer collaboration pattern where services interact directly via events, without central control over the sequence.

Service Composition

Combining multiple services to create higher-level business functionality, so a complete process can be assembled from parts built independently.

Loose Coupling

Services are designed to have minimal dependencies on each other, so one can change or fail without forcing changes elsewhere.

Service Granularity

The level of functionality encapsulated within a single service, and the trade-off between many small services and fewer larger ones.

Statelessness

A design preference that services not retain client state between calls, so any instance can serve any request.

Execution Mesh

Central orchestration layer managing agent lifecycle, routing, and coordination, carrying work between agents and the systems they act on.

Agent Gateway

A single entry point managing routing, authentication, and rate limiting for external systems and human users interacting with agents, and the place where access policy is applied.

Agentic Orchestrator

System that coordinates multiple agents to execute complex automated workflows, assigning work, tracking progress, and deciding what happens when an agent fails or escalates.

Agent Choreography

A peer-to-peer collaboration pattern where agents coordinate directly through shared events or signals, without a central orchestrator directing each step.

Agent Assembly (or Agent Build)

Combining multiple agents to create complex autonomous systems and workflows, assembling specialists into something that completes an end-to-end process.

Agent Autonomy

Agents operate independently with minimal dependencies, making autonomous decisions within their granted scope rather than waiting on a controller for every step.

Agent Scope

The breadth and depth of responsibilities assigned to a specific agent, which determines both what it can accomplish alone and how much of the system its failures reach.

Session State Boundary

An explicit definition of what an agent retains in operational memory between interactions and what it deliberately discards — since useful agents often need memory rather than none.

Idempotency

A property ensuring repeated calls produce the same effect as a single call, making retries safe after a timeout or partial failure.

Circuit Breaker

A resilience pattern that halts calls to a failing service to prevent cascading failure, resuming only once it recovers.

Event-Driven Architecture

Architecture where services respond to and produce events for loose coupling, with producers unaware of which consumers will react.

Message Exchange

Structured communication patterns between services, defining request/response, publish/subscribe, and the message shapes each side expects.

Service Integration Protocol

Standard protocols for web service communication and data exchange, such as SOAP and REST over HTTP.

Interoperability

Ability of different systems and services to work together effectively, across vendors, platforms, and data formats.

Enterprise Integration Patterns (EIP)

Standard patterns for integrating enterprise applications and services — named solutions to recurring routing, transformation, and messaging problems.

Canonical Data Model

A shared, standardized data schema used across services to avoid point-to-point translation between every pair of systems.

Idempotent Action

A property ensuring a repeated agent action produces the same effect as a single execution, which matters more here because retries are often decided autonomously.

Agent Fallback Protocol

A resilience pattern that halts delegation to a failing or underperforming agent and reroutes to a fallback agent or human escalation, rather than simply stopping.

Agent Triggers

Event-based activation mechanisms that prompt agents to take specific actions, so work begins when a condition is met rather than when someone invokes it.

Agent Communication Protocol (ACP)

Standardized methods for agents to exchange information and coordinate actions, covering not just message format but how intent, context, and results are conveyed.

Agent Integration Protocol (AIP)

Standardized interfaces and protocols for agent communication and integration, so agents built on different platforms work together without bespoke adapters.

Cross-Agent Standards

Protocols and standards enabling agents from different systems to collaborate, so a multi-agent workflow is not confined to one vendor's runtime.

Agent Patterns

Reusable design patterns for agent interaction, coordination, and integration, giving teams a shared name for structures they would otherwise reinvent.

Canonical Context Model

A shared, standardized schema for context entities so agents interpret the same situational data consistently across an enterprise, rather than each holding its own view.

Service Consumer / Provider

The two parties in a service interaction: the caller and the service being called.

Service Versioning

Managing multiple concurrent versions of a service interface as it evolves, so existing consumers keep working while new ones adopt the change.

Service Virtualization

Creating virtual representations of services for testing and development, so teams can build against dependencies that are unavailable or costly to call.

SLA (Service Level Agreement)

Service Level Agreement defining performance, availability, and quality metrics, and what happens when they are not met.

Service Level Objective (SLO)

An internal target metric that supports meeting an external SLA, set tighter than the commitment to leave room to recover.

Governance

Policies, processes, and controls for managing services throughout their lifecycle, from approval and deployment to retirement.

Requester / Fulfilling Agent

The two parties in an agent-fulfilled intent: whoever originates the request — a person, another agent, or an orchestrator — and whichever agent fulfills it.

Skill Versioning

Managing multiple concurrent versions of an agent's skill set or behavior as it evolves through feedback and memory updates, so changes are traceable rather than silent.

Agent Virtualization

Creating test agents by abstracting real-world conditions for testing, training, and development purposes, so behavior can be exercised safely before it touches production work.

Agent QoS

Quality of Service metrics defining agent performance, reliability, and response standards, including accuracy and escalation rates alongside latency and uptime.

Agent Performance Objective

An internal target metric supporting an agent's external QoS commitments, tracked continuously so degradation is caught before it breaches the commitment.

Agentic Governance

Framework for managing agent behavior, compliance, ethics, and lifecycle management, including audit trails, human oversight, and accountability for autonomous decisions.
